# BİL4010

## Paralel programlama

### 2.Hafta

### Paralel programlamaya Giriş

# İçerik

- Paralel programlama nedir?
- Neden paralel programlamaya ihtiyaç duyuyoruz?
- Paralel programların gereksinimi
- Paralel programlar
- Paralel sistemler

# Paralel Programlama Nedir?

- Paralel programlama (Paralel Hesaplama), bir bilgisayar donanımı üzerinde belirli işlemlerin dağıtılıp eş zamanlı olarak çalışması mantığına dayanmaktadır.
- Paralel bilgisayarlarda programlama için **işletim sistemi seviyesinde** ve/veya **programlama dili** seviyesinde pek çok yazılım sistemi geliştirilmiştir.
- Bu sistemler, problemin parçalara bölümlenmesini ve işlemcilere atanmasını sağlayan çeşitli mekanizmalar içermelidir.

# Seri Hesaplama

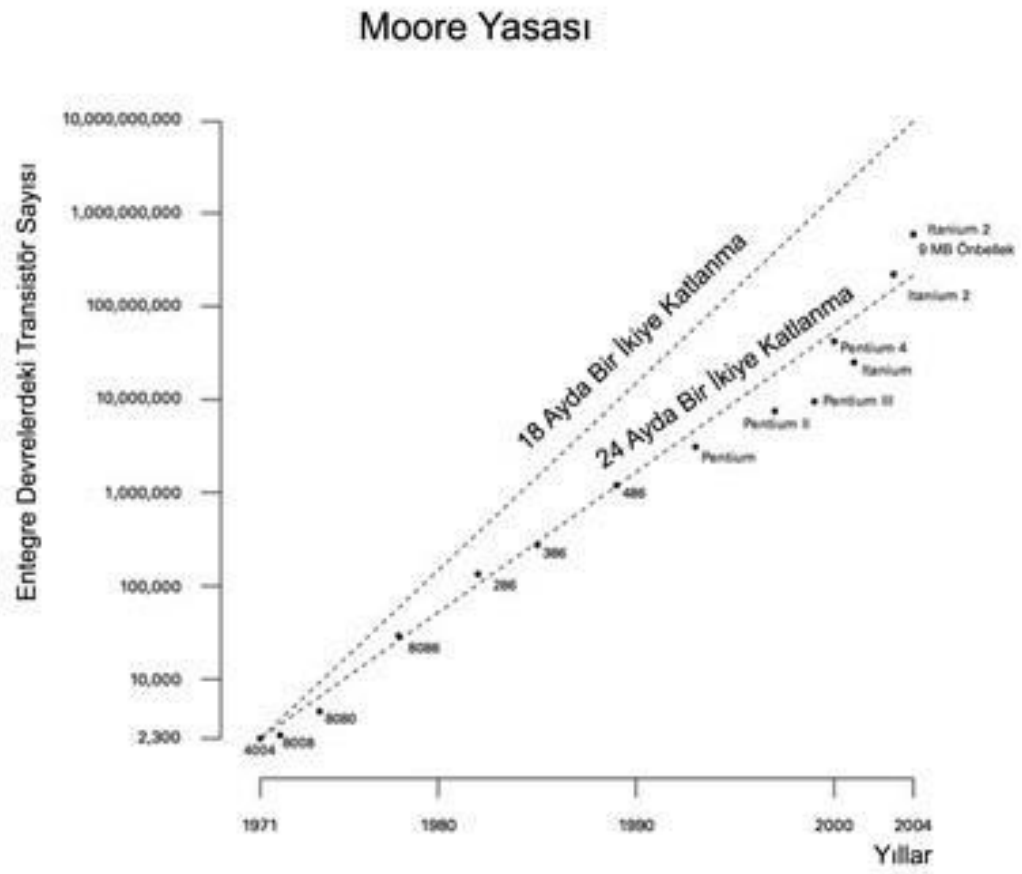
- Geleneksel olarak yazılımlar seri hesaplama için yazılır.
- Yazılım bilgisayar üzerindeki tek Merkezi İşlem Birimi (CPU) üzerinde çalışmaktadır.
- Problem komutlara ayrılmaktadır.
- Komutlar ardışık olarak çalıştırılmaktadır.
- Herhangi bir anda sadece bir komut çalıştırılabilmektedir.

# Paralel Hesaplama

- Bir problemi çözmek için çok sayıdaki hesaplama kaynağını aynı anda kullanmaktadır.
- Program çok sayıda CPU üzerinde çalışmaktadır.
- Problem **eşzamanlı** olarak çözülebilecek komutlara ayrılır.
- Her bir parça kümesi komutlara ayrılır.
- Her bir parçanın komutları aynı anda farklı CPU'lar üzerinde çalıştırılır.

# Moore Yasası

- Moore Yasası, Intel şirketinin kurucularından Gordon Moore'un 1965 yılında ortaya attığı bir yasa
- Her 18 ayda bir tümleşik devre üzerine yerleştirilebilecek bileşen sayısının iki katına çıkacağını, bunun bilgisayarların işlem kapasitelerinde büyük artışlar yaratacağını, üretim maliyetlerinin ise aynı kalacağını söylüyor.



# Paralel Programlamanın Ortaya Çıkışı

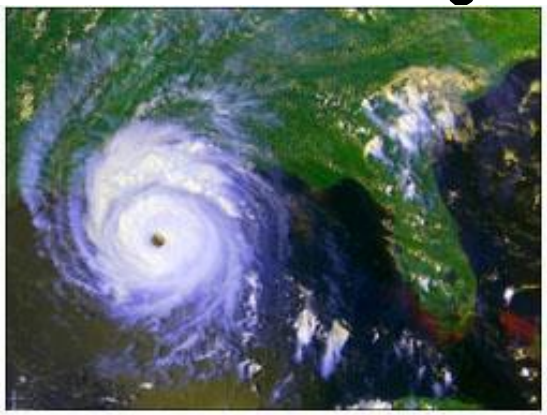
- 1996 ile 2002 yılları arasında işlemciler her yıl %50 oranında hızlanıyordu.
- Fakat, 2002 yılından sonra bu gelişme oranı %20 oranına düştü.
- 2005 yılında işlemci üreticileri işlemcilerin performanslarını artırmak için paralel işlemci mimarisine geçme kararı aldılar.
- Ancak bilgisayar üzerinde çalışan programların çoğu tek bir işlemci üzerinde çalışan seri programlardı ve istenilen performansı sağlamaktan çok uzaktaydılar.

# Paralel Programların Gereksinimi

- Aşağıdakilerinin seri bir yazılım yöntemi ile modellendiğini düşünün?

Galaxy Formation

Planetary Movments



Climate Change

Rush Hour Traffic

Plate Tectonics

Weather

# Paralel Programların Gereksinimi

- Günümüzde hesaplanması oldukça uzun süren problemlerin çözümlenmesi için paralel hesaplama yöntemlerine ihtiyaç duyulur
  - İnsan genomunun (gen şifrelerinin) çözümlenmesi,
  - Medikal görüntüleme sistemleri,
  - Hızlı ve tutarı arama motorlarının geliştirilmesi,
  - Kısa sürede ya da anlık trafik ve meteoroloji tahminlerinin hesaplanması,
  - Yapay zekaya sahip karar destek sistemleri veya gerçekçi oyun motorlarının kodlanması

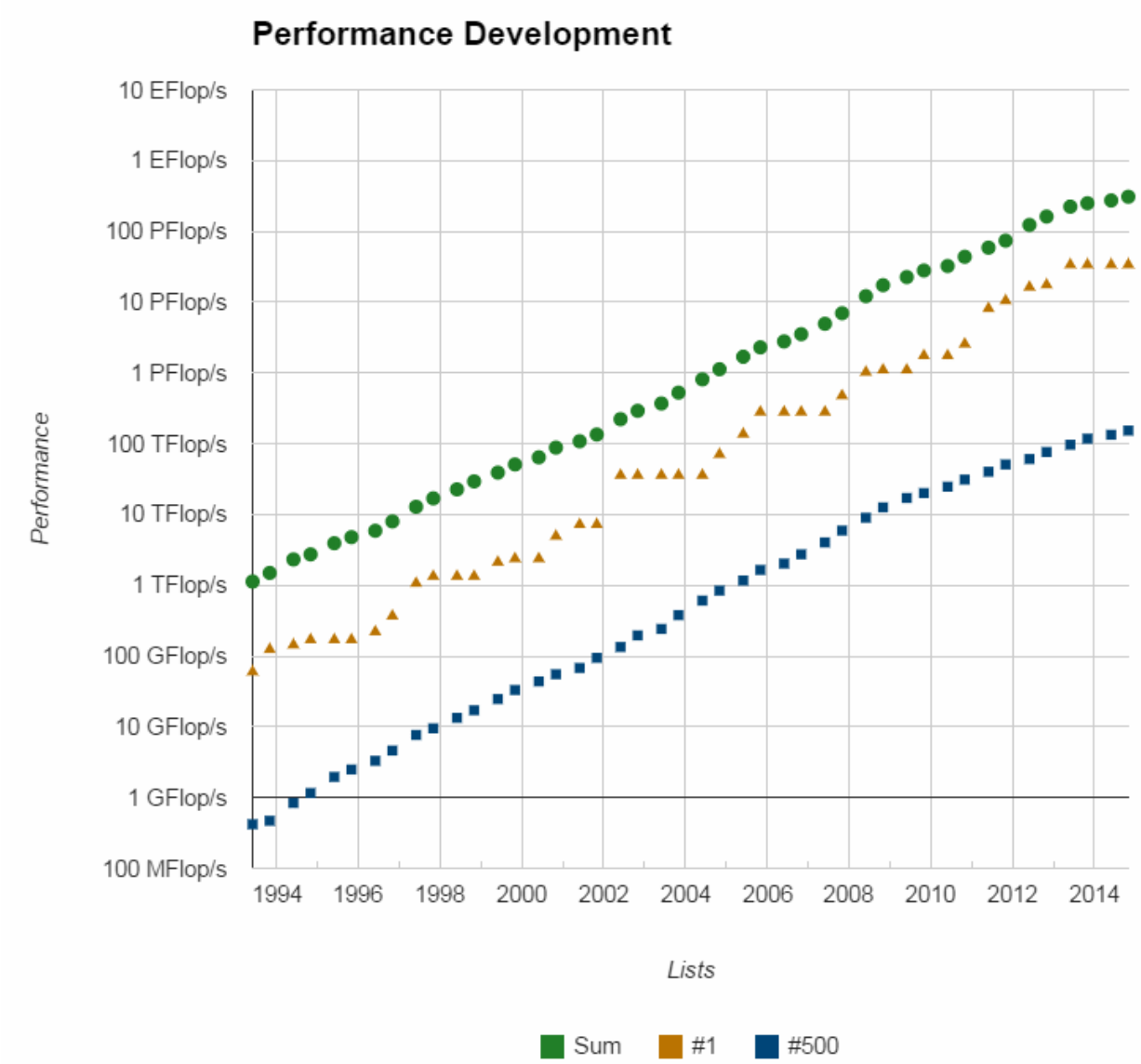
# Paralel Programlarının Faydaları

- Zaman ve paradan tasarruf
- Kompleks ve büyük ölçekli problemlerin çözümlenmesi
- Eşzamanlı olarak veri işlemek
- Problemi bilgisayar ağları ile dağıtarak problemleri farklı noktalarda çözmek
- Paralel donanımların daha iyi anlaşılması

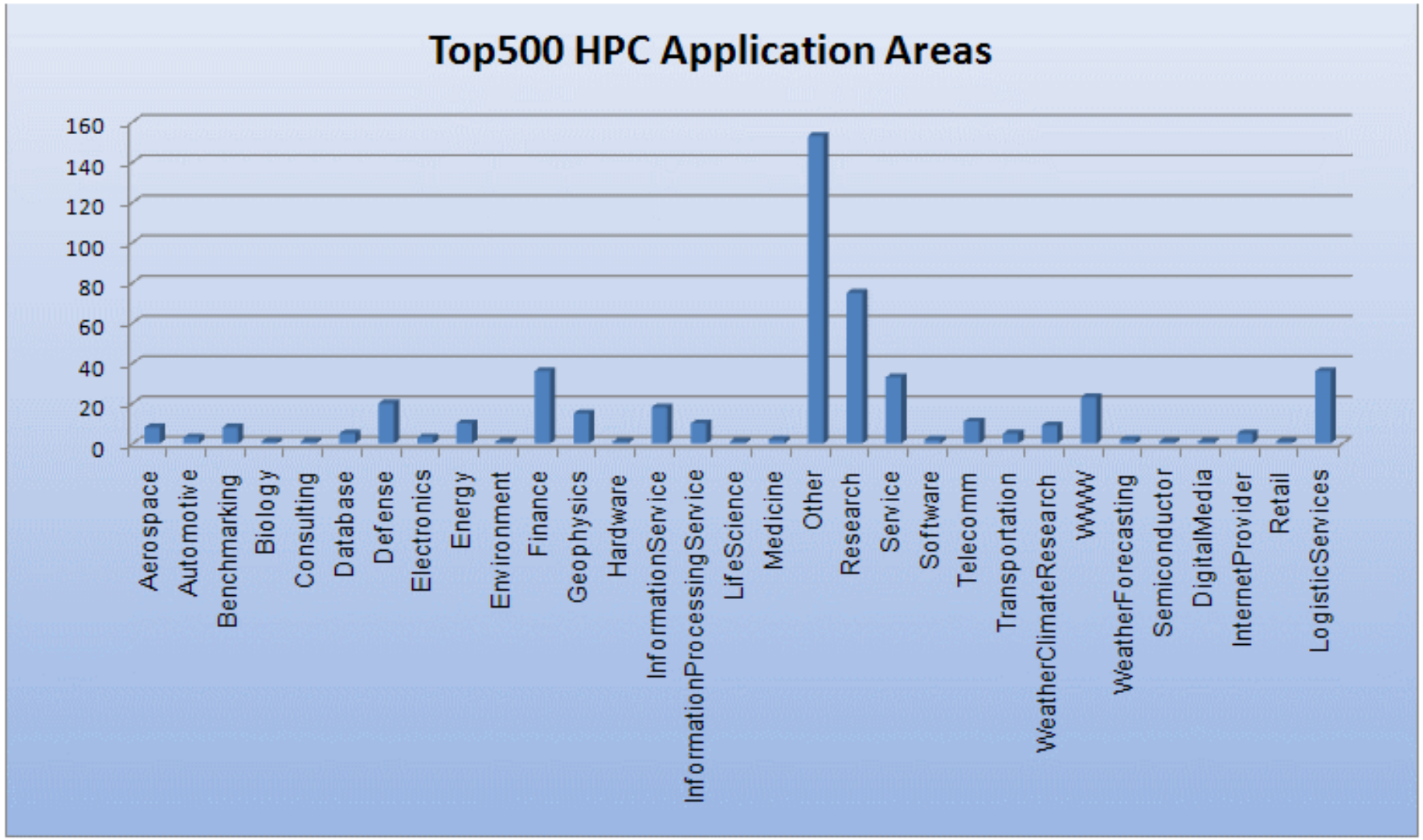
# İşlem Kapasitesinin Gelişimi

Exaflop

= $10^{18}$ calculations per second



# Paralel Programlama Kullanım Alanları



# Paralel Programlama Örneği

N adet değerin bir fonksiyon ile hesaplanarak toplanması

Klasik (Seri) yöntem ile:

```
sum = 0;
for (i = 0; i < n; i++) {
   x = Compute next value(. . .);
   sum += x;
}
```

# Paralel Programlama Örneği

Şimdi farz edelim ki *p* adet çekirdeğe (core) sahip bir bilgisayarımız olsun ve *p<<N* olduğu durumda:

```
my sum = 0;
my first_i = . . . ;
my last_i = . . . ;
for (my_i = my_first_i; my_i < my_last_i; my_i++) {
   my_x = Compute next value(. . .);
   my_sum += my_x;
}
```

**my_** ifadesi her bir core için çalışmaktadır.

# Paralel Programlama Örneği

- *N=24* ve *p=8* olduğu durumda bir seri içindeki değerlerini toplayalım
  - 1,4,3 9,28 5,1,1 6,2,7 2,5,0 4,1,8 6,51 2,3,9
  - Her bir core için bu değerlerin toplamı (**my_sum**):

| Core | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| my_sum | 8 | 19 | 7 | 15 | 7 | 13 | 12 | 14 |

  - Son olarak bir **master core** tüm core'lardan hesaplanan değerleri (my_sum) toplayarak toplam değerini hesaplamamız gerekli:

# Paralel Programlama Örneği

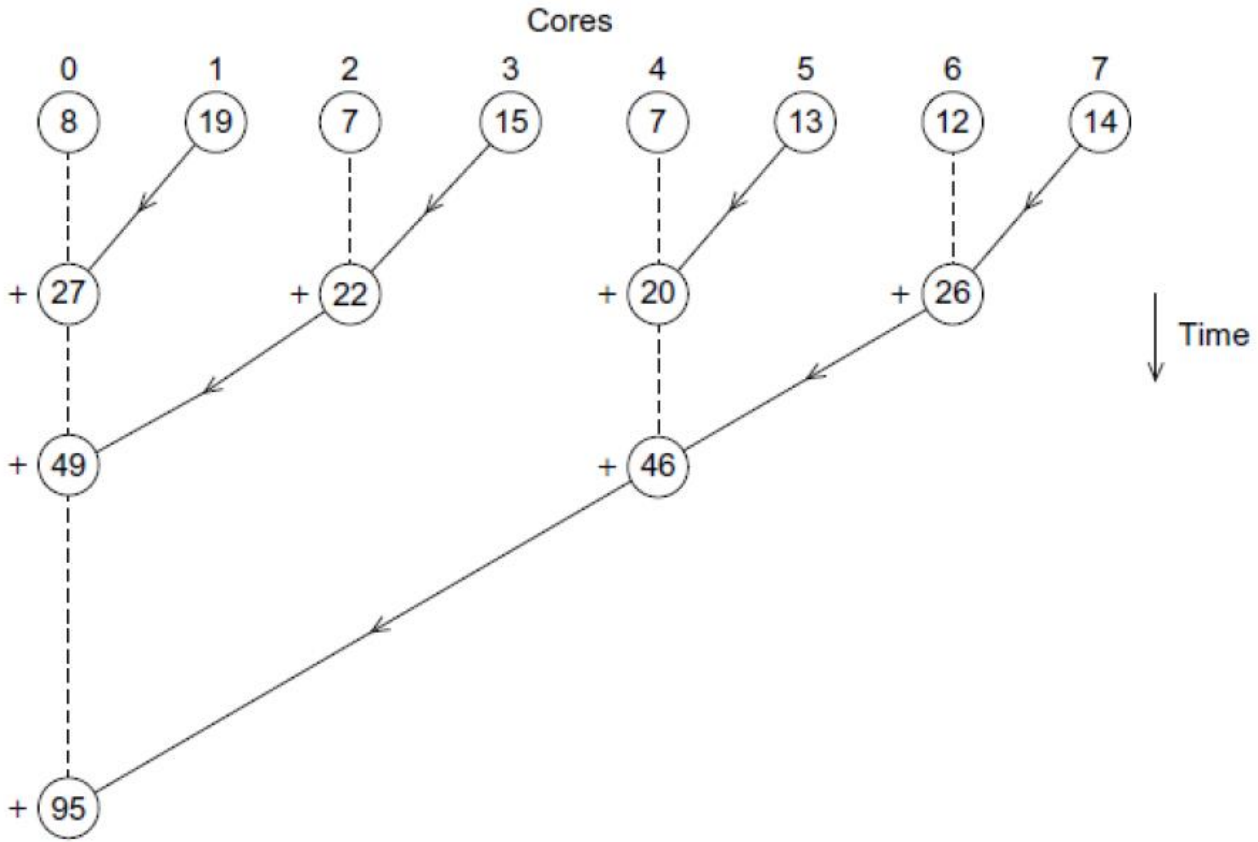
```
if (I'm the master core) {
    sum = my_x;
    for each core other than myself {
        receive value from core;
    sum += value;
    }
    else {
    send my_x to the master;
}
```

# Paralel Programlama Örneği

- Bu değeri de 8+19+7+15+7+13+12+14=95 olarak hesaplayabiliriz.
  - Bu yöntem ile master core 0 diğer 7 adet core'dan gelen veriyi toplayarak genel toplamı hesaplayacaktır.
- Bu işlem seri olarak yapılsaydı 24 adımda tamamlanacaktı.
- Paralel olarak yapıldığında bu görev 3+7=10 adımda tamamlanacaktır.
- Peki sistemi daha da hızlandırmak mümkün mü?

# Paralel Programlama Örneği

Mevcut sistem bir ağaç yapısına dönüştürülürse:




Sistemin işlemi tamamlaması 3+3=6 adımda sona erecektir.

# Paralel Programlama Örneği

- Her iki paralel yöntem arasındaki fark core sayısı arttıkça daha belirgin hale gelir:
  - Örn: 1000 core'a sahip olan bir sistemde birinci yöntemin işlemi tamamlaması için 999 adım sürmesine rağmen ikinci yöntem bu işlemi sadece 10 adımda tamamlayacak. (Neredeyse 100 kat daha hızlı)
- Paralel programların verimliliği senkronizasyon ve geliştirilen algoritmaya oldukça bağlıdır.

# Paralel Program Nasıl Yazılmalı

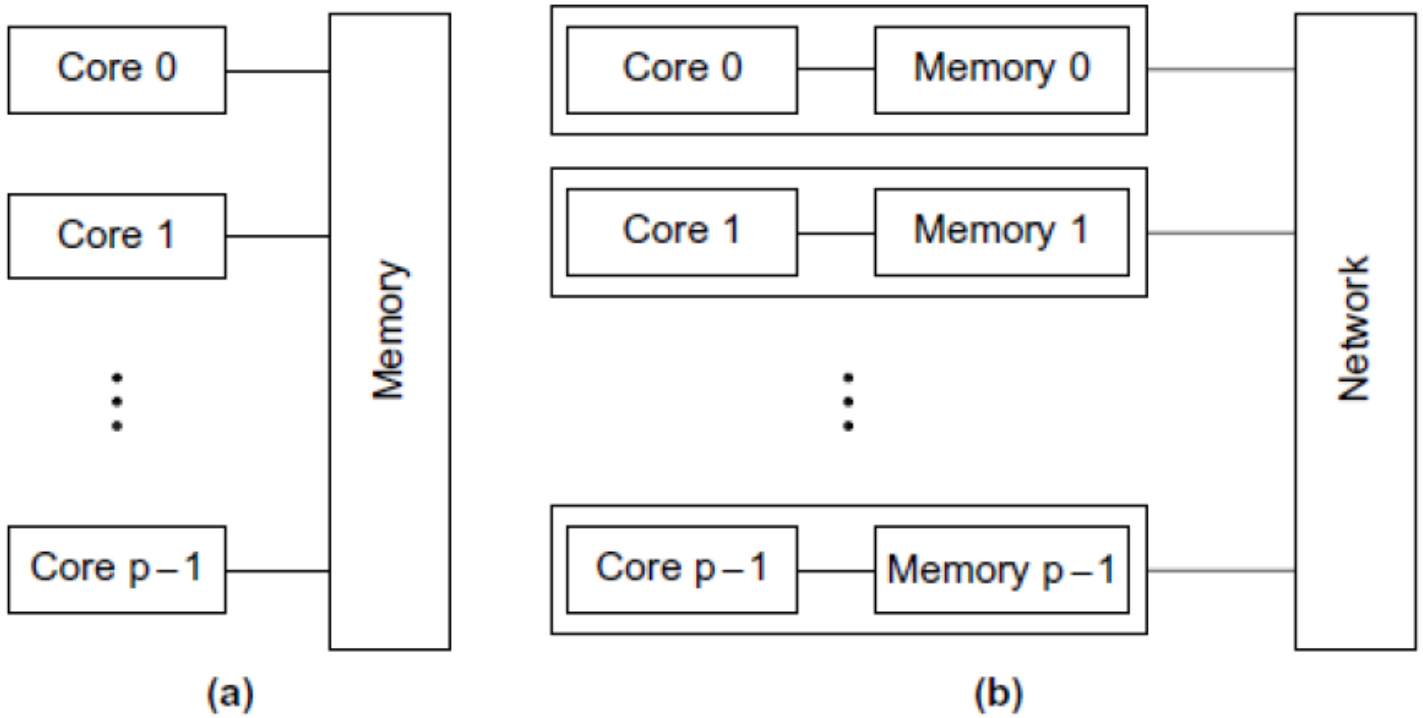
- Paralelleştirme algoritmaları temelde iki yönteme dayanmaktadır:

  Task-Parallelizm (İşlemlerin paralleştirilmesi)

  Data-Parallelizm (Veriyi paralel işleme)

# Paralel Sistemler

- Genel olarak paralel donanımları ikiye ayrılmaktadır:
  - a) Hafıza Paylaşımlı Sistemler (Shared Memory Systems)
  - b) Dağıtık Hafızalı Sistemler (Distributed Memory Systems)



Teşekkürler.

Sorular...